

Form Builder

User Guide

Build, preview, and fill dynamic forms — from a simple configuration

Version 1.0 · July 2026

Contents

1. Welcome — What Form Builder Does

2. Getting Started

3. Filling Out a Form (for Respondents)

4. Building a Form — The Basics

5. Field Types Reference

6. Validation — Making Sure Answers Are Correct

7. Field Widths & Responsive Layout

8. Multi-Step Forms (Wizards)

9. Field Dependencies — Show, Hide & Disable

10. OTP & Verification Fields

11. Previewing & Submitting

12. Tips & Troubleshooting

1. Welcome — What Form Builder Does

Form Builder turns a simple, written **form description** (a configuration) into a complete, polished, working web form — with labels, help text, validation, multi-step navigation, phone-number verification, and a responsive layout that adapts to phones, tablets, and desktops.

There is no drag-and-drop editor. Instead, you describe your form as a list of fields, and the app renders it for you. This makes forms easy to copy, review, version, and reuse across projects.

What you get out of the box

- **18 ready-made field types** — text, email, password, phone, date pickers, dropdowns, sliders, file uploads, OTP codes, and more.
- **Automatic validation** — required fields, length limits, patterns, password strength, date limits, and file-size checks, all with clear inline error messages.
- **Multi-step wizards** — split long forms into numbered steps with Back / Next navigation.
- **Smart dependencies** — show, hide, or disable a field based on another field's answer.
- **OTP verification** — send a one-time code to an email address or phone number and verify it before the form can be submitted.
- **Responsive widths** — place fields side by side on large screens and stack them on phones.
- **Dark mode and right-to-left (RTL) language support.**

2. Getting Started

Starting the app

From the project folder, start the development server, then open `http://localhost:3000` in your browser:

```
npm run dev      (or: yarn dev)
```

Pages you can visit

Page	Address	What it shows
Kitchen-Sink Demo	<code>/demo</code>	A three-step form using every field type. Includes Dark/Light and RTL/LTR toggle buttons, and shows the submitted answers as JSON below the form.
Example Forms Index	<code>/forms</code>	Links to the three realistic example forms below.
Live Account	<code>/forms/live-account</code>	A full KYC-style signup: name, nationality, date of birth, strong password, phone number, and a phone OTP.
Demo Account	<code>/forms/demo-account</code>	A minimal signup with a <i>chained</i> verification flow: verify your email first, then your phone.

IB Registration	/forms/ib	An Introducing Broker journey with a conditional field that only appears when relevant.
-----------------	-----------	---

Tip: In the example forms, the OTP verification is simulated — the code `123456` is always accepted, so you can try the whole flow without a real phone or inbox.

3. Filling Out a Form (for Respondents)

If you are answering a form built with Form Builder, here is what to expect:

- **Required fields** are marked with an asterisk (*) next to the label. You cannot submit — or move to the next step — until they are filled in correctly.
- **Error messages** appear directly under a field when something is wrong (for example, "Letters and spaces only" under a name field). Fix the value and the message disappears.
- **Help text** may appear under a field's label to explain what is expected.
- **Steps:** longer forms are split into numbered steps shown at the top (for example, *1 Account · 2 Profile · 3 Extras*). Use **Next** and **Back** to move between them. Your answers are kept when you go back.
- **Phone fields** include a country flag dropdown — pick your country, then type the number.
- **Date fields** open a calendar; some forms use a date *range* (start and end date).
- **Searchable dropdowns** let you type to filter long lists (for example, a country list).
- **File uploads** may restrict the file type (e.g. PDF only) and the maximum size in MB.
- **Repeating groups** (e.g. "Team members") have an *Add* button to add more entries and a remove button on each entry, within a minimum/maximum count.
- **Verification codes (OTP):** some forms ask you to verify your email or phone. See section 10 for the exact flow.

4. Building a Form — The Basics

A form is described by a single configuration object with an `id`, an optional `title` and `description`, a list of `fields`, and (optionally) `steps`. Each field needs at minimum a `type` and a unique `name`.

```

{
  id: "contact-us",
  title: "Contact us",
  fields: [
    { type: "text", name: "fullName", label: "Full name", required: true },
    { type: "email", name: "email", label: "Email", required: true },
    { type: "textarea", name: "message", label: "Message",
      placeholder: "How can we help?" },
    { type: "submit", name: "send", text: "Send message" },
  ],
}

```

Options every field understands

Option	What it does
<code>name</code>	Unique identifier; also the key used for the answer in the submitted data. No dots allowed.
<code>label</code>	The visible caption above the field.
<code>description</code>	Smaller help text shown with the field.
<code>placeholder</code>	Grey hint text shown inside an empty field.
<code>required</code>	Marks the field with * and blocks submission until answered.
<code>disabled</code>	Renders the field greyed out and uneditable.
<code>width</code>	How much of the row the field occupies — see section 7.
<code>visibleWhen</code> / <code>disabledWhen</code>	Conditions based on other fields — see section 9.
<code>enabledWhenVerified</code>	Keeps the field locked until a named OTP field is verified — see section 10.

Note: Every form should end with a `submit` field — that is the button users press to send their answers. In multi-step forms it automatically appears only on the last step.

5. Field Types Reference

Type	What it is	Key options
<code>text</code>	Single-line text input	<code>rules</code> (length, pattern, allowed characters)
<code>email</code>	Email input with format validation	<code>rules</code>
<code>password</code>	Password with show/hide toggle	<code>complexity</code> (uppercase, lowercase, number, special, minLength), <code>rules</code>

<code>textarea</code>	Multi-line text	<code>rules</code>
<code>number</code>	Numeric input	<code>min</code> , <code>max</code> , <code>step</code>
<code>otp</code>	One-time verification code boxes	<code>length</code> (e.g. 6), <code>dependsOn</code>
<code>phone</code>	International phone input with country flags	<code>defaultCountry</code> , <code>preferredCountries</code>
<code>select</code>	Dropdown; optionally searchable and/or multi-select	<code>options</code> , <code>searchable</code> , <code>multiple</code>
<code>radio</code>	Radio buttons — pick exactly one	<code>options</code>
<code>checkbox</code>	Single yes/no box, or a checkbox group	<code>options</code> (providing options makes it a group)
<code>switch</code>	On/off toggle	—
<code>date</code>	Calendar picker — single date or range	<code>range</code> , <code>minDate</code> , <code>maxDate</code> (format <code>yyyy-MM-dd</code>)
<code>slider</code>	Draggable value slider	<code>min</code> , <code>max</code> , <code>step</code>
<code>file</code>	File upload button	<code>accept</code> (e.g. ".pdf"), <code>maxSizeMB</code> , <code>multiple</code>
<code>hidden</code>	Invisible value carried along with answers (IDs, campaign tags)	<code>value</code>
<code>static</code>	Heading, paragraph, or divider — display only, no answer	<code>content</code> , <code>as</code> ("h1", "h2", "p", "divider")
<code>group</code>	Repeatable set of sub-fields ("Add another...")	<code>fields</code> , <code>min</code> , <code>max</code>
<code>submit</code>	The submit button	<code>text</code> , <code>variant</code> (default, outline, secondary, ...)

6. Validation — Making Sure Answers Are Correct

Validation is derived automatically from the field configuration — you never write it separately, so the form and its rules can never disagree.

Text rules

Text, email, password, and textarea fields accept a `rules` object:

Rule	Effect
<code>minLength</code> / <code>maxLength</code>	Minimum / maximum number of characters.
<code>pattern</code>	A pattern the value must match, e.g. <code>^[A-Za-z]+\$</code> .
<code>message</code>	The friendly error to show when the pattern fails, e.g. "Letters and spaces only".
<code>allow</code>	Characters permitted while typing (e.g. <code>"A-Za-z "</code>) — anything else is simply blocked as the user types.
<code>trim</code>	Strips leading/trailing spaces before validating and before submitting.

Password strength

Password fields can additionally demand complexity. Each unmet requirement is reported to the user until the password qualifies:

```
{ type: "password", name: "password", label: "Create Password", required: true,
  complexity: { uppercase: true, lowercase: true, number: true,
               special: true, minLength: 8 } }
```

Other built-in checks

- **Email** — must be a valid email address.
- **Phone** — must be a valid number for the selected country.
- **Number / slider** — kept within `min` / `max` (and `step`).
- **Date** — kept within `minDate` / `maxDate` (e.g. a date-of-birth field capped so applicants are 18+).
- **File** — checked against `accept` file types and `maxSizeMB`.
- **OTP** — must be fully entered *and successfully verified* (section 10).
- **Group** — must contain between `min` and `max` entries, and every entry's own fields are validated too.

Where errors appear: always inline, directly beneath the offending field. In multi-step forms, pressing **Next** validates only the current step's fields and moves focus to the first problem it finds.

7. Field Widths & Responsive Layout

Fields are laid out on a grid. Each field can take the **full** row, or share it: **half**, **third**, or **quarter**. If you set nothing, a field is full width.

One width everywhere

```
{ type: "number", name: "age", label: "Age", width: "half" }
```

Different widths per device

Screens are grouped into three breakpoints. Any breakpoint you leave unset falls back to full width:

Breakpoint	Screen width	Typical device
mobile	up to 480 px	Phones
tablet	481 – 1024 px	Tablets
desktop	1025 px and up	Laptops / desktops

```
// Stacked (full width) on phones, side by side from tablet up:
{ type: "text", name: "firstName", label: "First Name", required: true,
  width: { tablet: "half", desktop: "half" } },
{ type: "text", name: "lastName", label: "Last Name", required: true,
  width: { tablet: "half", desktop: "half" } }
```

Tip: This "full on mobile, half from tablet up" pattern is the most common — it keeps phone layouts readable while making desktop forms compact. Two `half` fields, three `third` fields, or four `quarter` fields fill one row.

8. Multi-Step Forms (Wizards)

To split a form into steps, add a `steps` list. Each step has a `title` and the `fieldNames` it contains:

```
steps: [
  { title: "Account", fieldNames: ["firstName", "email", "password"] },
  { title: "Profile", fieldNames: ["newsletter", "size", "color"] },
  { title: "Extras", fieldNames: ["birthday", "cv", "team"] },
]
```

How steps behave

- A numbered **step indicator** appears at the top; the current step is highlighted and completed steps are marked.
- **Next** validates only the current step. If something is invalid, the user stays put and focus jumps to the first field with an error.
- **Back** is always available (except on the first step) and never loses entered answers.
- The **Submit** button appears only on the last step, in place of Next.
- **Hidden fields** travel along automatically no matter which step they are listed in.
- Verified OTP fields **stay verified** when you navigate away and back — users are not asked to re-verify just for revisiting a step.

9. Field Dependencies — Show, Hide & Disable

Fields can react to other fields' answers using declarative conditions — no code required.

Show a field only when relevant (`visibleWhen`)

```
{ type: "radio", name: "existingIb", label: "Are you an existing IB?",
  options: [ { label: "Yes", value: "yes" }, { label: "No", value: "no" } ] },
{ type: "text", name: "currentBroker", label: "Current broker name",
  required: true,
  visibleWhen: { field: "existingIb", equals: "yes" } }
```

The broker-name field appears only after the user answers "Yes". While hidden, it is skipped by validation and its value is *not* included in the submission — a hidden required field never blocks submitting.

Disable instead of hide (`disabledWhen`)

```
{ type: "switch", name: "newsletter", label: "Newsletter" },
{ type: "text", name: "nickname", label: "Nickname",
  disabledWhen: { field: "newsletter", equals: false } }
```

The field stays visible but greyed out until the condition stops matching.

Condition operators

Operator	Matches when the other field...
<code>equals</code>	has exactly this value
<code>notEquals</code>	has any other value
<code>in</code>	has one of the listed values, e.g. <code>in: ["a", "b"]</code>

10. OTP & Verification Fields

An `otp` field displays individual code boxes (typically 6) and manages the whole send-and-verify flow.

What the user experiences

1. The user fills in the field being verified — for example their phone number.
2. The **Send** button unlocks only once that value is valid, and the code boxes stay locked until a code has actually been sent.
3. After sending, a **resend countdown** (30 seconds by default) starts; when it reaches zero the user may request a fresh code.
4. As soon as the last digit is typed, the code is **verified automatically** — no extra button press.
5. On success the field locks in a verified state. On failure an error appears and the user can correct the code or resend.
6. The form cannot be submitted until every required OTP field is verified.

Tying the code to what it verifies (`dependsOn`)

```
{ type: "phone", name: "phone", label: "Phone Number",
  defaultCountry: "AE", required: true },
{ type: "otp", name: "otp", label: "Verification Code (OTP)",
  length: 6, required: true, dependsOn: "phone" }
```

With `dependsOn`, the OTP is bound to the phone number it was sent to. If the user edits the phone number afterwards — even from another step — the code boxes clear and verification resets, so a code can never "stand in" for a number it was not sent to.

Chaining verifications (`enabledWhenVerified`)

You can gate any field behind a completed verification. The Demo Account form verifies email first, then unlocks the phone field, then verifies the phone:

```
{ type: "email", name: "email", required: true },
{ type: "otp", name: "emailOtp", length: 6, required: true,
  dependsOn: "email" },
{ type: "phone", name: "phone", required: true,
  enabledWhenVerified: "emailOtp" },
{ type: "otp", name: "otp", length: 6, required: true,
  dependsOn: "phone" }
```

Note: Keep an OTP field and the field it depends on in the *same step* of a multi-step form, so users can see and fix the value they are verifying.

11. Previewing & Submitting

Previewing your form

- Open your form's page in the browser while the dev server runs — changes to the configuration appear on refresh.
- The `/demo` page is the best playground: it prints the **submitted answers as JSON** right below the form, so you can confirm exactly what data your form produces.
- Use the demo page's **Dark/Light** and **RTL/LTR** buttons to preview both themes and right-to-left layouts.
- Resize the browser window (or use your browser's device toolbar) to check the mobile, tablet, and desktop widths from section 7.

What happens on submit

- Every visible field is validated one final time; submission is blocked until all errors are resolved.
- Hidden (`hidden` -type) values are included automatically; fields hidden by `visibleWhen` are excluded.
- Text values with `trim` are submitted trimmed.

- The answers are delivered as one clean object keyed by field `name` — e.g. `{ "firstName": "Ada", "email": "ada@example.com", ... }` — ready to send to your backend.

12. Tips & Troubleshooting

Symptom	Likely cause & fix
The form shows an error instead of rendering	The configuration is invalid — the app validates configs strictly (bad patterns, dotted field names, wrong date formats like anything other than <code>yyyy-MM-dd</code> , unknown country codes). Check the message and fix the offending field.
"Next" does nothing	A field on the current step is invalid; look for the inline error (focus jumps to it automatically).
Cannot submit even though everything looks filled	A required OTP field is not yet <i>verified</i> — entering the digits is not enough; verification must succeed.
OTP boxes suddenly cleared themselves	The value it depends on (email/phone) was edited, so the old code was invalidated on purpose. Send a new code.
Send button on the OTP is greyed out	The field it depends on is empty or invalid; fix that field first.
A field never shows up	Check its <code>visibleWhen</code> condition — and in a stepped form, make sure its <code>name</code> is listed in a step's <code>fieldNames</code> .
Two fields will not sit side by side	Give both a fractional <code>width</code> (e.g. <code>"half"</code>) for the breakpoint you are viewing; remember unset breakpoints fall back to full width.
Certain characters cannot be typed	The field has an <code>allow</code> rule that blocks them by design (e.g. letters-only name fields).